

Comparing Fitness Functions for Evolving Generalist Agents Using Genetic Algorithms

Evolutionary Computing 2024 - Standard Assignment - Task 2 - Group 12

Sarra Ben Ltaief
Vrije Universiteit Amsterdam
Student Number: 2830336
Amsterdam, Netherlands
s.benltaief@student.vu.nl

Nika Kovačić
Vrije Universiteit Amsterdam
Student Number: 2862120
Amsterdam, Netherlands
n.kovacic@student.vu.nl

Clàudia Domènech Farré
Vrije Universiteit Amsterdam
Student Number: 2742397
Amsterdam, Netherlands
c.domenech.farre@student.vu.nl

Emad Mshalwat
Vrije Universiteit Amsterdam
Student Number: 2658370
Amsterdam, Netherlands
e.mshalwat@student.vu.nl

ACM Reference Format:

Sarra Ben Ltaief, Clàudia Domènech Farré, Nika Kovačić, and Emad Mshalwat. 2024. Comparing Fitness Functions for Evolving Generalist Agents Using Genetic Algorithms: Evolutionary Computing 2024 - Standard Assignment - Task 2 - Group 12. In *Proceedings of October 20, 2024*. ACM, New York, NY, USA, 4 pages. <https://doi.org/10.1145/nnnnnnn.nnnnnnn>

1 INTRODUCTION

The EvoMan framework allows users to test optimization algorithms by training an agent with multiple objectives, namely a generalist agent that can fight against multiple enemies in the same training phase [1]. One approach to improving the performance of these agents is through evolution, where evolutionary algorithms optimize neural network architectures and their associated parameters. The goal of the genetic algorithms (GAs) in this study is to optimize the weights of a neural network controller, which influences the behaviour of agents as they face multiple enemies. An important element of this optimization process is the design of fitness functions, which provide metrics for evaluating agent performance.

The EvoMan framework simulates various combat scenarios and different fitness functions can influence the performance of generalist agents. This is the basis of our research question:

RQ: How do different fitness function designs impact the win rate of generalist agents in the EvoMan framework?

The first fitness function, which is the default in the EvoMan framework, has a balanced approach by considering energy levels, player performance, and time taken to defeat enemies. On the other hand, the second fitness function prioritizes speed, imposing penalties for longer combat durations. By comparing two different GAs with alternative fitness functions, we aim to showcase how fitness functions influence the success of agents and their ability to perform in various combat situations.

2 METHODS

2.1 Evolutionary Concepts

As neuroevolutionary methods are built using evolutionary algorithms, the first step in this research is to establish the fundamental ideas of an evolutionary algorithm. The method begins with a population, in which each individual has a fitness value that varies as the population evolves. Selection initiates the cyclic evolutionary process. The selection mechanism for individuals to participate in the evolutionary process creates a database of parental genomes. Recombination happens between two randomly selected parental genomes, resulting in the exchange of incomplete information from their genomes to produce new offspring. Furthermore, as with natural evolution, an offspring may occasionally acquire genes that are not inherited from either parent. As a result, the offspring have the opportunity to alter portions of their genomes, introducing minor differences. The fitness values of the new offspring are assessed before further selection is conducted to choose individuals for the next generation. When the termination condition is met, i.e. after a certain number of generations, the algorithm ends.

2.1.1 Fitness Functions: Our research aims to investigate how different fitness function designs impact the win rate of generalist agents in the EvoMan framework. To this end, we implement two versions of a Genetic Algorithm (GA) that differ in their fitness functions.

GA1 Fitness Function. The default fitness function in the EvoMan framework is given by:

$$f = 0.9 \cdot (100 - e) + 0.1 \cdot p - \log(t) \quad (1)$$

Where e and p represent the energy levels of the enemy and player, respectively, ranging from 0 to 100, and t represents the time taken to defeat the enemy. The fitness criterion used is the maximization of fitness, meaning the optimal solution is to obtain the greatest possible fitness. The selection of individuals, as described in the last section, is fitness-based, and the highest values are selected.

GA2 Fitness Function. The second fitness function modifies the default to prioritize speed, penalizing agents that take longer to defeat their enemies:

$$f = 100 - e - k \cdot t \quad (2)$$

Where k is a constant controlling the time penalty. By adjusting k , we can vary the emphasis on the time taken to defeat the enemy. This fitness function encourages agents to defeat enemies quickly, potentially at the expense of the player's remaining energy.

2.2 Parameters and Experimental Setup

In our research, since we are interested in the possible effects of alternative fitness function designs on a neuroevolutionary algorithm's ability to generalize, we develop two GA methods that employ different fitness functions so that we can compare the performance of each method. Both methods make use of neural networks, each with one hidden layer made up of ten neurons.

To build a model with consistent performance, we must tune numerous parameters in our algorithms. To obtain an optimal model, the parameters utilized in the algorithms are fine-tuned by conducting ten trials with the selected hyperparameter values and calculating the average fitness score. In this way, we base our conclusions on the assumption that our model architecture is best suited for investigating the impact of various fitness function designs. Table 1 shows the final parameters employed in our genetic algorithms. Our generalist agent (AI player) is trained against different enemies using the same hyperparameter parameters. The first training group includes opponents 3, 6, and 7, while the second group includes enemies 4, 7, and 8 from the EvoMan framework. One run for such a group entails optimizing and evolving the networks across 30 generations. At the end of each generation, the mean and maximum fitness levels are recorded. At the end of all 30 generations, the best genome is saved and tested against all 8 enemies to measure overall performance. This is repeated five times and averaged to allow for potential stochasticity. One such run is performed independently ten times. Fitness trajectories from several generations are averaged. The results across runs for the winner genomes are not averaged but instead used to create a distribution of optimal individual gains. Finally, we keep track of the absolute optimal winner across training groups and runs. The evolutionary algorithm utilizes classic evolutionary components. Selection is made by tournament selection, with a tournament size of 5. Reproduction is defined as a single-point crossover, in which parents exchange genetic material to produce children. To preserve diversity, a 0.2 mutation rate is applied following crossover, resulting in random alterations to the genetic material. Elitism is practiced by maintaining the top two

persons from each generation to ensure that the best solutions are not overlooked.

Table 1: Parameters used in the Genetic Algorithms

Parameter	Value
Enemies	(3, 6, 7)(4, 7, 8)
Population size	100
Generations	30
Mutation rate	0.2
Tournament size	5
Elitism size	2
Number of hidden neurons	10
Runs	10

3 RESULTS

The results of our experiments comparing the two fitness functions of GA1 and GA2 will be compared in this section. We are focusing on the agent’s performance across the two enemy groups ([3, 6, 7] and [4, 7, 8]).

3.1 Fitness Progression

First, for Enemy Group [3, 6, 7], GA1 showed higher mean and maximum fitness scores compared to GA2 (Figure 1). Both algorithms converged around generation 20, but GA1 stabilized at a higher fitness level. GA2 had slower fitness growth and reached a lower plateau, which means it had a less efficient optimization process. Secondly, in Enemy Group [4, 7, 8], GA1 again slightly outperformed GA2 (Figure 1), maintaining higher mean and maximum fitness throughout 30 generations. GA2 showed a slower improvement and higher variability in fitness scores across the generations.

3.2 Individual Gain Analysis

For Enemy Group [4, 7, 8], GA1 had more stable individual gains, with a median of around 55 and a smaller range, indicating a more steady performance (Figure 1). However, GA2 showed more variation, with agents performing worse at times, but other times achieving results similar to GA1’s. For Enemy Group [3, 6, 7], GA1 was again more consistent, with most gains around 60, while GA2 was less predictable but occasionally performed better, despite the wider variation in its results.

3.3 Best Agent Performance

In Table 2, we show the energy values (for player and enemy) for the best-performing agents across the enemies. There, we can see that GA1 consistently had higher player energy and lower enemy energy, especially against enemies 3, 6 and 7, confirming the algorithm’s superior ability to balance offensive and defensive strategies. GA2’s best agents tended to focus more on speed, as evidenced by the lower enemy energy, but this often came at the expense of player survival, as shown by the reduced player energy.

3.4 Statistical testing - Win Rate Analysis

We ran t-tests to compare the win rates of GA1 and GA2 across both enemy groups. The results show no significant difference in win rates between the two algorithms. On the one hand, for the enemy group [4, 7, 8], the p-value was 0.999, meaning no significant difference. On the other hand, for the enemy group [3, 6, 7], the p-value was 0.0637, which also shows no significant difference. From these results we can see that although GA1 had a more consistent performance overall, both GA1 and GA2 had similar success (in terms of defeating the enemies). Therefore, while GA1’s balanced strategy was more reliable, GA2 could still perform well despite the variability.

Table 2: Player and Enemy Energy Levels and Gain

Enemy	Player Energy	Enemy Energy	Gain (Player - Enemy)
1	21.27	0.00	21.27
2	88.97	44.00	44.97
3	84.21	0.00	84.21
4	21.41	0.00	21.41
5	89.65	52.60	37.05
6	75.13	0.00	75.13
7	84.69	0.40	84.29
8	84.35	5.20	79.15

4 DISCUSSION

The results demonstrated that GA1’s balanced fitness function slightly outperformed GA2’s time-penalty-based approach in the majority of cases. In this section, we’ll look at why these differences exist and what they mean for evolutionary algorithms.

4.1 Fitness Function Impact

GA1’s better performance comes from its balanced evaluation because it takes into account enemy energy, player survival, and time. This made agents try out more strategies, causing quicker improvements and more stable results. The smaller variation in fitness and individual gains shows us that GA1 helped agents develop strategies that worked more consistently across all the runs.

However, GA2 showed less consistent results because it focuses more on the speed itself. With the time penalty GA2 makes agents prioritize rapid eliminations, but it also means that in some cases agents would not last long in the game. As a consequence, the strategies are more aggressive but less effective, (because the agents had a harder time balancing between beating the enemy and staying alive). The higher variability in individual gains shows us that GA2 agents were in general less consistent, performing incredibly well at times but failing in others.

4.2 Adaptability Across Enemy Groups

We saw that GA1 consistently performed well across both enemy groups, making agents that could handle different types of attack patterns and strategies effectively. For example, GA1 performed extremely well against enemies like Bubbleman (Enemy 7), where it was important to balance defense and offense to handle the sinusoidal projectiles. On the other hand, GA2 had more varied results.

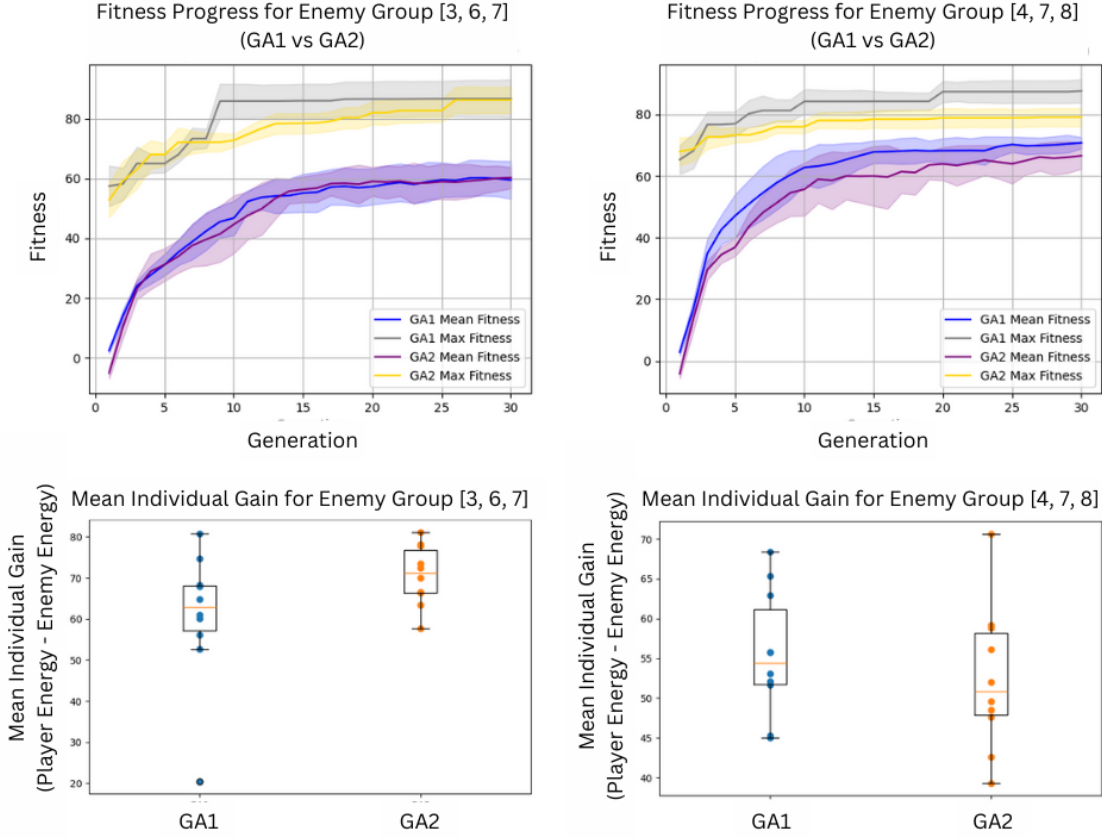


Figure 1: The top row shows the fitness progress over 30 generations for enemy groups [3, 6, 7] on the left and [4, 7, 8] on the right. The bottom row presents box plots of the mean individual gain which is the difference between player energy and enemy energy for the same enemy groups.

It performed better against enemies with simpler and more predictable attack patterns, like for example, Crashman (Enemy 6), where it was essential to avoid close combat and timing jumps. However, GA2 struggled against faster, more dynamic enemies like Heatman (Enemy 4) and Bubbleman (Enemy 7) which required a more strategic defense. The time penalty in GA2 pushed agents towards way more aggressive strategies that were effective in simpler situations but also failed when more complex behaviors were needed. Overall, GA2 worked well for specific, less complicated enemies, but GA1’s balanced approach was significantly more successful across the different types of enemies.

4.3 Implications and Future Work

Our findings suggest that designing fitness functions for generalist agents requires a balance between optimizing for speed and maintaining consistent performance. We saw that GA2’s time-penalty approach still lacks reliability for a more generalist agent design. Future research could explore hybrid fitness functions that mix the strengths of both approaches, adjusting the balance between speed and survival dynamically based on the evolution phase or the specific enemy. Adaptive fitness functions could also improve agent

performance by adjusting criteria in response to agent behaviour, potentially leading to more flexible and adaptable strategies.

5 CONCLUSIONS

In this paper, we investigated how different fitness function designs affected the win rate of generalist agents in the EvoMan framework. We examined the effects of two genetic algorithms on agent performance: GA1, which has a balanced fitness function that takes into account opponent energy, player survival, and time, and GA2, which prioritizes speed by penalizing longer conflicts.

Our studies showed that the difference in the GAs is not significant. GA1 slightly outperformed GA2, however, a more thorough analysis could lead to different results. A balanced fitness function that takes into account numerous dimensions of performance promotes the development of more robust and adaptable tactics.

REFERENCES

- [1] Karine da Silva Miras de Araújo and Fabrício Olivetti de França. 2016. An electronic-game framework for evaluating coevolutionary algorithms. *arXiv preprint arXiv:1604.00644* (2016).